PROJECT DOCUMENTATION

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

COLLEGE NAME	ARASU ENGINEERING COLLEGE, KUMBAKONAM.
TEAM LEADER	AADHAVA A anandaadhava31@gmail.com
TEAM MEMBER 1	AARYAN JAIN N jainy263@gmail.com
TEAM MEMBER 2	HARISH J jayapalpal946@gmail.com
TEAM MEMBER 3	VISHAL M vishalmathi08@gmail.com
TEAM MEMBER 4	VIGNESH S svignesh2342006@gmail.com
PLATFORM	SALESFORCE CRM / SALESFORCE DEVELOPER ORG

Prepared from the supplied WhatNext Vision Motors project specification and implementation record.

CONTENTS. Documentation Structure

1. Project Overview

2. Use Case and Business Objectives

3. Requirements Analysis

4. Salesforce Solution Architecture

5. Data Model: Objects, Fields and Relationships

6. Lightning App and User Navigation

7. Automation Flow 1: Dealer Assignment

8. Automation Flow 2: Test Drive Reminder

9. Apex Trigger Handler and Stock Control

10. Batch Apex and Scheduled Apex

11. End-to-End Operational Workflow

12. Implementation Milestones

13. Validation and Testing Scenarios

14. Benefits, Constraints and Implementation Notes

15. Conclusion

16. Future Enhancement Areas

17. Appendix: Source Code and Configuration Summary

Source terminology note

The supplied document uses both “WhatNext” and “WhatsNext” in the narrative. This report uses the project title supplied for the documentation: “WhatNext Vision Motors”.

1. Project Overview

WhatNext Vision Motors is a Salesforce CRM implementation for an automotive business focused on improving customer experience and streamlining operational processes. The project centers on vehicle management, dealer information, customer records, vehicle orders, test drives and service requests. The solution combines Salesforce data modelling, Lightning App configuration, record-triggered flows, Apex trigger handling, Batch Apex and Scheduled Apex.

The project specification describes three major operational outcomes: assign an appropriate dealer based on customer location, prevent orders for vehicles with no available stock, and automate order-status processing when inventory changes. A separate reminder flow supports scheduled test drives by notifying customers before their appointment.

Project focus

- Customer-centric vehicle ordering and dealer assignment.
- Accurate and controlled vehicle stock management.
- Automated communication for scheduled test drives.
- Bulk processing of pending vehicle orders after stock changes.
- A maintainable Salesforce implementation using declarative automation together with Apex.

Learning outcomes documented in the source

- Data Modelling
- Fields and Relationships
- Lightning App Builder
- Record Triggered Flows
- Apex and Apex Triggers
- Batch Apex
- Scheduled Apex

2. Use Case and Business Objectives

The supplied use case describes an automotive organization that wants a more convenient ordering experience, better stock accuracy and lower administrative effort. Customers should be guided toward an appropriate dealer, prevented from ordering vehicles that are unavailable, and kept informed about order status and test-drive appointments.

Business objectives

Objective	How the project addresses it
Improve customer experience	Uses customer information, dealer information and automated reminders to reduce manual steps.
Control vehicle availability	Checks stock before order placement and reduces stock when confirmed orders are processed.
Improve order transparency	Uses Pending, Confirmed, Delivered and Canceled order

Objective	How the project addresses it
	statuses.
Reduce operational effort	Uses Flow, Apex and scheduled processing instead of relying on manual updates.
Support service interactions	Maintains dedicated test-drive and service-request records linked to customer and vehicle.

Primary actors / records

- Customer: maintains customer profile, contact information, address and preferred vehicle type.
- Dealer: maintains dealer identity, location and contact information.
- Vehicle: maintains model, price, stock quantity, dealer relationship and availability status.
- Vehicle Order: represents a purchase request and its fulfillment status.
- Test Drive: represents a scheduled/completed/canceled driving appointment.
- Service Request: records service date, issue description and service status.

3. Requirements Analysis

3.1 Functional requirements

ID	Requirement
FR-01	Store and manage vehicle details and stock availability in Salesforce.
FR-02	Store dealer details and connect dealers with vehicle/order processes.
FR-03	Store customer information, including address and preferred vehicle type.
FR-04	Track vehicle orders with customer, vehicle, date and status.
FR-05	Prevent order placement when the selected vehicle has no stock.
FR-06	Assign a dealer using customer location information.
FR-07	Track scheduled test drives and send a reminder one day before the test drive.
FR-08	Track vehicle service requests and their status.
FR-09	Run batch processing so pending orders can be re-evaluated when stock becomes available.
FR-10	Run the batch processing on a schedule using Scheduled Apex.

3.2 Non-functional expectations derived from the implementation approach

- Maintainability: business logic is placed in a dedicated trigger-handler class rather than being embedded entirely in the trigger.
- Automation: routine assignment, reminders and batch processing are performed by Salesforce automation.
- Data consistency: stock validation and stock decrement are centralized in Apex logic.
- Operational visibility: the Lightning App provides navigation across the core objects and reporting/dashboard entry points.

4. Salesforce Solution Architecture

The documented solution uses Salesforce as the central application platform. Custom objects store the automotive domain data, the Lightning App provides the operational workspace, Flow handles event-driven automation, and Apex handles transaction-sensitive stock logic and scheduled bulk processing.

Layer	Documented component	Role
Platform	Salesforce CRM / Developer Org	Hosts the complete implementation.
Data model	Six custom objects	Stores vehicles, dealers, customers, orders, test drives and service requests.
User interface	Lightning App: WhatNext Vision Motors	Provides navigation to the business objects, Reports and Dashboard.
Declarative automation	Record-triggered Flow	Dealer assignment and test-drive reminder.
Programmatic logic	VehicleOrderTriggerHandler	Validates stock and decrements stock for confirmed orders.
Bulk processing	VehicleOrderBatch	Re-evaluates pending vehicle orders after inventory changes.
Scheduling	VehicleOrderBatchScheduler	Invokes the batch job on a schedule.

High-level flow

Customer / staff creates or updates business records → Salesforce evaluates Flow / Trigger logic → Vehicle order is validated against stock → Dealer assignment and communication automation run where applicable → Confirmed orders reduce stock → Scheduled Batch Apex revisits pending orders → system remains synchronized with available inventory.

5. Data Model: Objects, Fields and Relationships

The project defines six custom objects. The supplied source explicitly documents their intended purpose and key fields. The following table consolidates that model into a single reference.

Object API Name	Purpose	Key relationship / role
Vehicle__c	Stores vehicle details, pricing, stock and status.	Lookup to Vehicle_Dealer__c; referenced by Orders, Test Drives and Service Requests.
Vehicle_Dealer__c	Stores authorized dealer information.	Dealer/location record used during order assignment.
Vehicle_Customer__c	Stores customer identity, contact and address.	Referenced by Orders, Test Drives and Service Requests.
Vehicle_Order__c	Tracks vehicle purchases and fulfillment state.	Lookup to Vehicle_Customer__c and Vehicle__c.
Vehicle_Test_Drive__c	Tracks test-drive bookings.	Lookup to Vehicle_Customer__c and Vehicle__c.
Vehicle_Service_Request__c	Tracks vehicle servicing requests.	Lookup to Vehicle_Customer__c and Vehicle__c.

Key fields

Object	Documented fields
Vehicle__c	Vehicle Name (Text); Vehicle_Model__c (Picklist: Sedan, SUV, EV, etc.); Stock_Quantity__c (Number); Price__c (Currency); Dealer__c (Lookup); Status__c (Picklist: Available, Out of Stock, Discontinued).
Vehicle_Dealer__c	Dealer_Name__c (Text); Dealer_Location__c (Text); Dealer_Code__c (Auto Number); Phone__c; Email__c.
Vehicle_Order__c	Vehicle_Customer__c (Lookup); Vehicle__c (Lookup); Order_Date__c (Date); Status__c (Pending, Confirmed, Delivered, Canceled).
Vehicle_Customer__c	Customer_Name__c; Email__c; Phone__c; Address__c; Preferred_Vehicle_Type__c.
Vehicle_Test_Drive__c	Vehicle_Customer__c; Vehicle__c; Test_Drive_Date__c; Status__c (Scheduled, Completed, Canceled).
Vehicle_Service_Request__c	Vehicle_Customer__c; Vehicle__c; Service_Date__c; Issue_Description__c; Status__c (Requested, In Progress, Completed).

Relationship summary

Parent / source	Related record	Relationship described in source
Vehicle_Customer__c	Vehicle_Order__c	Lookup reference on the order
Vehicle__c	Vehicle_Order__c	Lookup reference on the order

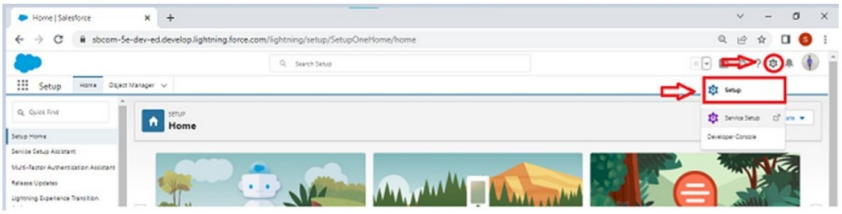
Parent / source	Related record	Relationship described in source
Vehicle_Customer__c	Vehicle_Test_Drive__c	Lookup reference on test drive
Vehicle__c	Vehicle_Test_Drive__c	Lookup reference on test drive
Vehicle_Customer__c	Vehicle_Service_Request__c	Lookup reference on service request
Vehicle__c	Vehicle_Service_Request__c	Lookup reference on service request
Vehicle_Dealer__c	Vehicle__c	Vehicle contains Dealer__c lookup

MileStone 2 :

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:
Click on gear icon → click setup.



Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.

Figure 1. Source project screenshot showing the documented object model and relationships.

6. Lightning App and User Navigation

The documented Lightning App is named “WhatNext Vision Motors”. The source configures it through App Manager and adds the custom business objects to the navigation menu. The documented navigation items are Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports and Dashboard.

Navigation item	Purpose
Vehicle	View and manage vehicle inventory records.
Dealer	View and manage dealer records.
Customer	View and manage customer records.
Order	Create and track vehicle purchase orders.
Test Drive	Create and manage test-drive appointments.
Service Request	Create and track service requests.
Reports	Provide access to Salesforce reports configured for the app.
Dashboard	Provide access to a dashboard configured for the app.

Configuration reference

The source instructs the user to add the custom objects to the Lightning App and make the app available to the System Administrator profile.

7. Automation Flow 1: Dealer Assignment

The first flow is described as a record-triggered flow for assigning a dealer to an order based on customer location. The flow runs when a Vehicle_Order__c record is created and the order status is Pending.

Step	Documented configuration
Trigger	Record-Triggered Flow on Vehicle_Order__c; record created; Status__c = Pending.
Get customer	Get Records on Vehicle Customer where Id equals $Record.Vehicle_Customer__c.
Get dealer	Get Records on Vehicle Dealer where Dealer_Location__c equals the customer Address__c.
Update	Update Records using the dealer record returned by Get Nearest Dealer.
Activation	Flow label: Auto Assign Dealer; flow is activated.

Business interpretation

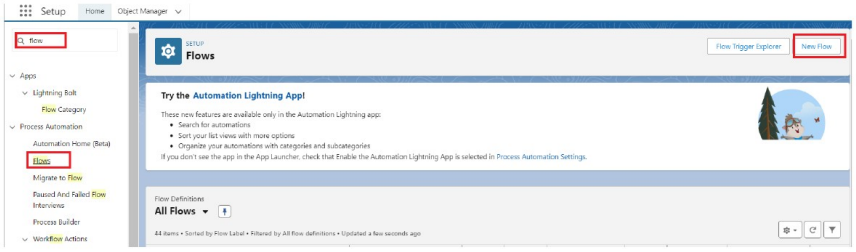
The intended business outcome is to connect a newly created pending order to an appropriate dealer. The source describes this as “nearest dealer” assignment. In the documented implementation steps, the lookup condition compares Dealer_Location__c directly with the customer Address__c. Therefore, the current configuration should be understood as a location-value match rather than a demonstrated geographic-distance calculation.

Activity 4 : Creating Remaining Objects and Fields.

MileStone 6 :

Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer’s location

Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Triggered Automation From Flow.

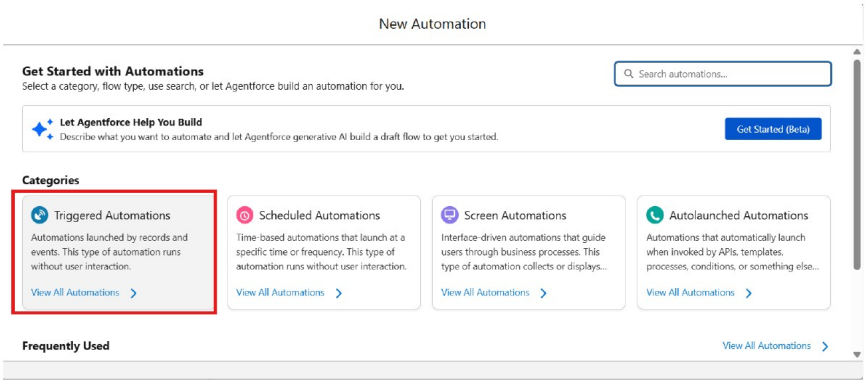


Figure 2. Source screenshot of the Flow creation step for dealer assignment.

8. Automation Flow 2: Test Drive Reminder

The second flow automates an email reminder for customers who have a scheduled test drive. The flow is record-triggered on Vehicle_Test_Drive__c and includes a scheduled path one day before Test_Drive_Date__c.

Element	Documented configuration
Trigger	Vehicle_Test_Drive__c; record created or updated; Status__c = Scheduled.
Scheduled path	Reminder Before Test Drive; time source = Test_Drive_Date__c; offset = 1 day before.
Customer lookup	Get customer record using the order/test-drive customer reference.
Email action	Send Email action labelled Send Test Drive Reminder.
Subject	Reminder: Your Test Drive is Tomorrow!
Recipient	Customer email returned from Get Customer Information.
Flow label	Test Drive Reminder; saved and activated.

The source also includes a screenshot of the resulting email and the Flow configuration. This provides implementation evidence for the automated customer communication use case.

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True

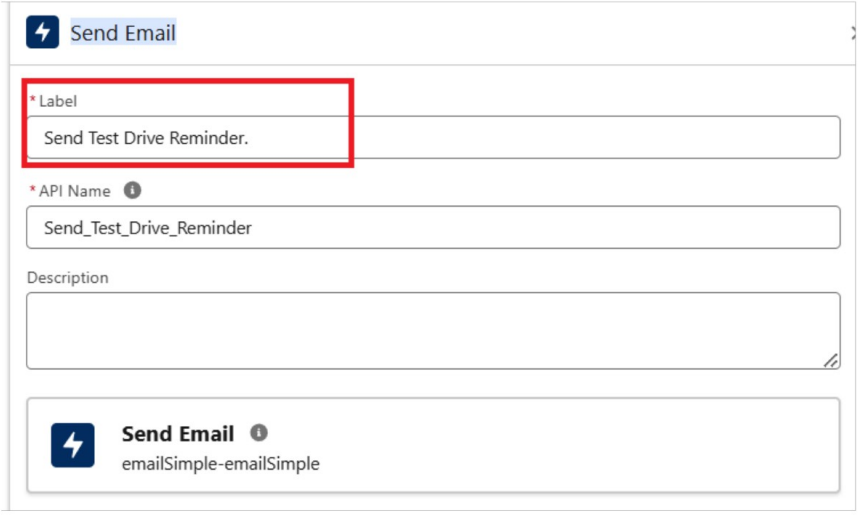


Figure 3. Source screenshot showing the Send Email configuration for the test-drive reminder.

9. Apex Trigger Handler and Stock Control

The Apex implementation introduces a dedicated class named VehicleOrderTriggerHandler. The documented handler separates “before” validation from “after” stock updates. This structure is intended to keep trigger logic modular and maintainable.

Execution context	Documented action
Before insert / update	Call preventOrderIfOutOfStock(newOrders).
After insert / update	Call updateStockOnOrderPlacement(newOrders).
preventOrderIfOutOfStock	Collect vehicle IDs, query Vehicle__c stock quantity, and add an error when Stock_Quantity__c <= 0.
updateStockOnOrderPlacement	For Confirmed orders, collect vehicle IDs, query stock, decrement Stock_Quantity__c by one and update vehicles.

Stock validation behavior

The source code uses order.addError("This vehicle is out of stock. Order cannot be placed.") when the selected vehicle has a stock quantity of zero or less. This prevents the order transaction from proceeding for unavailable inventory.

Stock decrement behavior

For orders whose Status__c is Confirmed, the handler reduces Stock_Quantity__c by one when stock is greater than zero. The updated vehicle records are collected and persisted in a single update operation.

```
public class VehicleOrderTriggerHandler {
    public static void handleTrigger(
        List<Vehicle_Order__c> newOrders,
        Map<Id, Vehicle_Order__c> oldOrders,
        Boolean isBefore,
        Boolean isAfter,
        Boolean isInsert,
        Boolean isUpdate) {

        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders);
        }
        if (isAfter && (isInsert || isUpdate)) {
            updateStockOnOrderPlacement(newOrders);
        }
    }

    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) vehicleIds.add(order.Vehicle__c);
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [
                SELECT Id, Stock_Quantity__c
                FROM Vehicle__c
                WHERE Id IN :vehicleIds
            ]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
                }
```

```
            }
        }
    }
}
```

Listing 1. Condensed excerpt from the supplied VehicleOrderTriggerHandler implementation.

10. Batch Apex and Scheduled Apex

The project includes a batch-processing scenario for pending orders. The documented business sequence is: a customer creates an order when stock is unavailable; the order remains Pending; when stock is later replenished, the batch job re-evaluates the pending order and updates it to Confirmed.

Scheduled Apex

The supplied source includes the scheduler class VehicleOrderBatchScheduler. It creates a VehicleOrderBatch instance and calls Database.executeBatch with a batch size of 50.

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Listing 2. Scheduler class supplied in the project source.

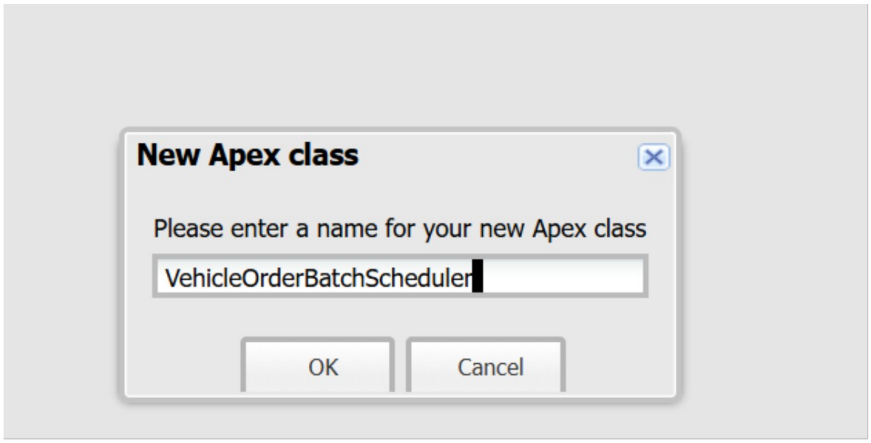
Scheduling expression documented in the source

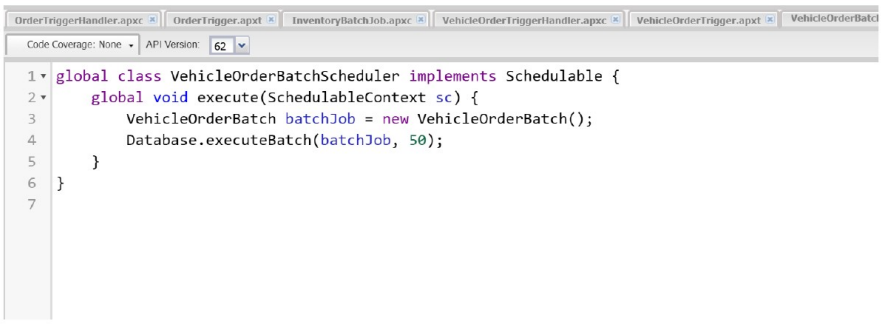
```
String cronExp = '0 0 12 * * ?';
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

Important source note

The project text describes this as a nightly batch job in one instruction, while the supplied cron expression is commented as daily at 12:00 PM. This report preserves the supplied expression and notes the discrepancy instead of silently changing it.

Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class





Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {
   global void execute(SchedulableContext sc) {
      VehicleOrderBatch batchJob = new VehicleOrderBatch();
      Database.executeBatch(batchJob, 50); // 50 is the batch size
   }
}
```

Figure 4. Source screenshot showing VehicleOrderBatchScheduler and batch size 50.

11. End-to-End Operational Workflow

Stage	Process
1. Customer setup	Customer details including address and preferred vehicle type are stored in Vehicle_Customer__c.

Stage	Process
2. Vehicle selection	Vehicle record contains model, price, stock quantity, dealer and status.
3. Order creation	Vehicle_Order__c is created with customer, vehicle, order date and status.
4. Dealer assignment	Pending orders trigger the documented dealer-assignment flow.
5. Stock validation	VehicleOrderTriggerHandler checks stock before order completion.
6. Confirmation	For a confirmed order, stock quantity is reduced by one.
7. Pending order recovery	If an order remains pending because stock was unavailable, VehicleOrderBatch is intended to revisit it after replenishment.
8. Test drive reminder	Scheduled test-drive records generate a reminder email one day before the test drive.
9. Service lifecycle	Vehicle_Service_Request__c records requested, in-progress and completed service states.

Process flow at a glance

CUSTOMER → VEHICLE SELECTION → VEHICLE ORDER → DEALER ASSIGNMENT → STOCK CHECK → CONFIRMED / PENDING → STOCK UPDATE → BATCH RECHECK OF PENDING ORDERS → FULFILLMENT. In parallel, TEST DRIVE → ONE-DAY REMINDER EMAIL and SERVICE REQUEST → REQUESTED / IN PROGRESS / COMPLETED.

12. Implementation Milestones

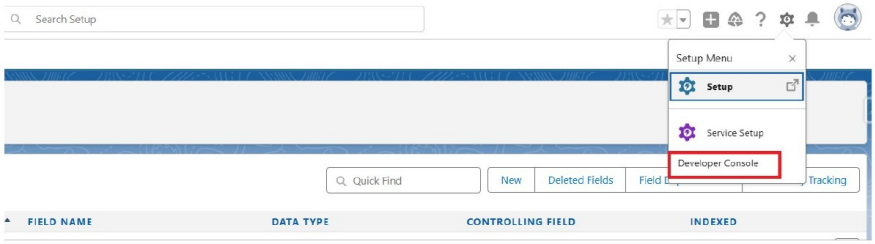
Milestone	Main activities documented
Milestone 1	Create Salesforce Developer Org and activate the account.
Milestone 2	Create Vehicle and Dealer objects, then create the remaining domain objects.
Milestone 3	Create custom tabs for the objects.
Milestone 4	Create and configure the WhatNext Vision Motors Lightning App.
Milestone 5	Create fields and relationships for vehicles, dealers, customers, orders, test drives and service requests.
Milestone 6	Create the dealer assignment Flow and the test-drive reminder Flow.
Milestone 7	Create Apex trigger handler, trigger, Batch Apex and

Milestone	Main activities documented
	Scheduled Apex.

MileStone 7 :

Create Apex and Trigger Batch Jobs :

Step 1 : Click Developer Console From Gear icon



Step 2 : click File then Select New then Select Apex.

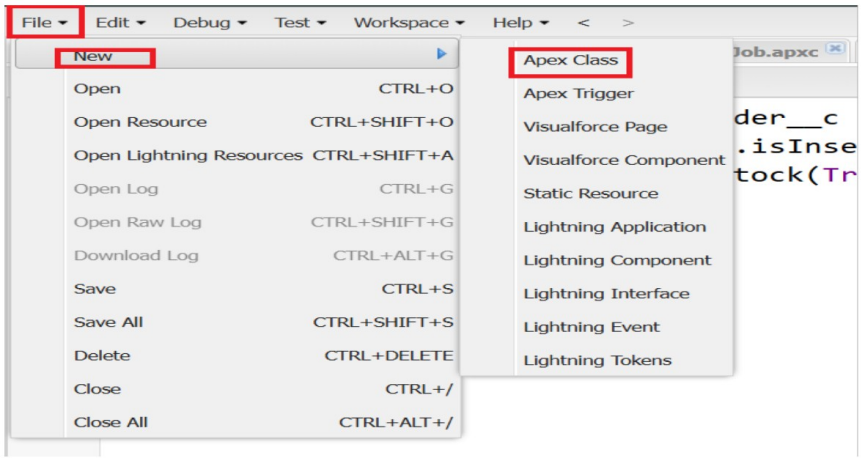


Figure 5. Source evidence for the Apex / Batch implementation milestone.

13. Validation and Testing Scenarios

The supplied project source demonstrates the implementation through configuration screenshots and an email test. It does not provide a formal pass/fail test matrix. The following scenarios are

therefore documented as validation scenarios derived directly from the implemented requirements rather than as unverified test results.

Test ID	Scenario	Expected behavior from project design
T01	Create order for vehicle with stock > 0	Order should be allowed to proceed; a confirmed order reduces stock by one.
T02	Create order for vehicle with stock = 0	Order should be blocked with the documented out-of-stock error.
T03	Create pending order and later replenish stock	Scheduled batch processing should revisit the order and confirm it when stock becomes available.
T04	Create pending order with customer and dealer information	Dealer-assignment flow should execute using the documented customer-address / dealer-location condition.
T05	Schedule test drive	A reminder email should be sent one day before the test-drive date.
T06	Run scheduler	VehicleOrderBatchScheduler should invoke VehicleOrderBatch with batch size 50.

Evidence note

The supplied source contains screenshots showing the Flow configuration, reminder email, Apex code and scheduler. It does not record formal test-case pass/fail outcomes.

14. Benefits, Constraints and Implementation Notes

Benefits

- Customer-oriented ordering workflow with automation around dealer selection.
- Stock validation reduces the risk of orders being created against unavailable vehicles.
- Confirmed orders automatically reduce inventory through Apex.
- Pending orders can be reprocessed after stock replenishment using batch automation.
- Test-drive reminders reduce manual customer follow-up.
- The Lightning App groups the core automotive records in a single workspace.

Constraints visible in the supplied implementation

- The documented dealer-selection condition is an equality match between dealer location and customer address; a geographic-distance calculation is not shown in the supplied steps.
- The formal batch class implementation is referenced by the scheduler and described behaviorally, but its full source code is not included in the supplied document text.

- The source includes a schedule comment describing “nightly at midnight” while the supplied cron expression is commented as 12:00 PM.
- Some field labels/API references vary in the instructional text (for example, “Vehicle Modal”, “Stock Quality”, and a customer reference shown as Customer__c in one Flow step). These should be standardized before production deployment.

Recommended production hardening based on the documented gaps

- Replace exact address matching with a deterministic dealer-distance or region-matching method.
- Standardize field/API names across all Flow, Apex and object configuration references.
- Add an explicit automated test suite for bulk orders, partial stock, repeated trigger execution and batch reprocessing.
- Define an unambiguous scheduler cadence and align the cron expression with the business requirement.
- Add monitoring/error handling for failed Flow, Apex and scheduled jobs.

15. Conclusion

WhatNext Vision Motors demonstrates how Salesforce can be used to model automotive sales and after-sales operations while reducing manual work through declarative and programmatic automation. The solution brings vehicles, dealers, customers, orders, test drives and service requests into a connected data model. The documented Flow, Apex trigger handler, Batch Apex and Scheduled Apex components work together around three practical business needs: dealer assignment, stock control and customer communication.

The project also provides a practical learning path from Salesforce Developer Org setup through custom objects, tabs, Lightning App configuration, fields and relationships, Flow automation and Apex development. Based strictly on the supplied material, the implementation is best viewed as a working project blueprint with several production-hardening opportunities identified above.

16. Future Enhancement Areas

- Geographic nearest-dealer calculation using coordinates, distance or mapping integration rather than exact text matching.
- Dealer capacity and stock-aware routing so that orders are assigned to the most suitable available dealer.
- Real-time inventory synchronization with external dealer or ERP systems.
- Customer-facing order-status notifications for Pending, Confirmed, Delivered and Canceled states.
- Automated stock-replenishment notifications for low-inventory vehicles.
- Expanded reporting and dashboards for inventory, dealer workload, test drives and service requests.
- Comprehensive Apex unit tests and regression scenarios for bulk order processing.
- Role-based access and field-level security aligned with dealer, sales and service responsibilities.

17. Appendix: Source Code and Configuration Summary

A. Core automation components

Component	Documented configuration / responsibility
Auto Assign Dealer	Record-triggered Flow on Vehicle_Order__c for Pending orders.
Test Drive Reminder	Record-triggered Flow with a scheduled path one day before Test_Drive_Date__c.
VehicleOrderTriggerHandler	Before-context stock validation and after-context stock decrement for confirmed orders.
Vehicle Order Trigger	Calls VehicleOrderTriggerHandler for Vehicle_Order__c insert/update events.
VehicleOrderBatch	Re-evaluates pending orders after inventory changes (behavior described; full source not present in the supplied document).
VehicleOrderBatchScheduler	Schedulable wrapper that executes VehicleOrderBatch with batch size 50.

B. Scheduler source supplied by the project

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

C. Scheduling command supplied by the project

```
String cronExp = '0 0 12 * * ?';
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

D. Project artefact references

The original project file contains step-by-step Salesforce configuration screenshots for Developer Org creation, custom objects, tabs, Lightning App setup, field creation, Flow configuration, email automation, Apex class creation, trigger creation and Scheduled Apex. Selected images are reproduced in this documentation as implementation evidence.

END OF DOCUMENTATION